

Lab 13 Report

Symbol Table Manager



Name: Ahmad Bin Rashid

Roll No: 2023-CS-53

Instructor: Ma'am Aliya

Course: Compiler Construction Lab

University of Engineering and Technology

1 Overview

This report documents the test cases executed against the Symbol Table Manager (`src/symbol_table.py`) and the integrated Pascal parser (`src/pascal_parser.py`).

Five test cases are presented, covering both valid programs and programs that contain the two semantic errors the compiler is required to detect:

Error Code	Description
E1	Duplicate declaration — same name declared twice in the same scope
E2	Use of undeclared identifier

2 Test Cases

2.1 Test Case 1 — Valid: Arithmetic Program

File: `test/test1_valid_arith.pas`

Description:

A well-formed Pascal program that declares three variables (`x`, `y`, `result`) and performs arithmetic in the body.

Expected behaviour:

- `x` and `y` are inserted as `integer`, `result` as `real`.
- All identifier uses in the body resolve successfully.
- No errors reported.

Actual output (excerpt):

```
[Scope 0] Enter
  [Scope 0] insert valid_arith : function, integer, line 2
[Scope 1] Enter
  [Scope 1] insert x           : variable, integer, line 4
  [Scope 1] insert y           : variable, integer, line 5
  [Scope 1] insert result      : variable, real, line 6
  [Scope 1] lookup x           -> found at scope 1
  [Scope 1] lookup y           -> found at scope 1
  [Scope 1] lookup result      -> found at scope 1
  [Scope 1] lookup x           -> found at scope 1
  [Scope 1] lookup y           -> found at scope 1
[Scope 1] Exit, dump:
+-----+-----+-----+-----+-----+-----+
| ID | Name   | Kind      | Type    | Scope | Line |
+-----+-----+-----+-----+-----+-----+
|  1 | x      | variable  | integer |    1  |    4  |
|  2 | y      | variable  | integer |    1  |    5  |
|  3 | result | variable  | real    |    1  |    6  |
+-----+-----+-----+-----+-----+-----+

PARSE + SYMBOL TABLE OK
```

Result: PASS

2.2 Test Case 2 — Valid: Control Flow

File: test/test2_valid_control.pas

Description:

A program using `if-else` and `while` with three declared variables.

Expected behaviour:

- All variables declared correctly; all uses resolve.
- No errors.

Actual output (excerpt):

```
[Scope 1] insert a    : variable, integer, line 4
[Scope 1] insert b    : variable, integer, line 5
[Scope 1] insert i    : variable, integer, line 6
...
      PARSE + SYMBOL TABLE OK
```

Result: PASS

2.3 Test Case 3 — Invalid: Duplicate Declaration

File: test/test3_dup_decl.pas

Description:

The variable `count` is declared twice in the same scope.

Expected behaviour:

- First declaration of `count` succeeds.
- Second declaration triggers **E1**.
- Error message includes line number of the offending declaration.
- Parser continues (error recovery) and reports remaining lookups correctly.

Actual output:

```
[Scope 1] insert count : variable, integer, line 5
[Scope 1] insert total : variable, real,      line 6
ERROR line 7: duplicate declaration 'count' in this scope

      FINISHED WITH 1 ERROR(S)
```

Result: PASS — E1 correctly detected on line 7.

2.4 Test Case 4 — Invalid: Undeclared Identifier

File: test/test4_undecl_id.pas

Description:

The expression `undeclared_var + x` uses a name that was never declared.

Expected behaviour:

- `x` and `y` are inserted successfully.
- Lookup of `undeclared_var` fails, triggering **E2**.

Actual output:

```
[Scope 1] lookup undeclared_var -> NOT FOUND
ERROR line 9: undeclared identifier 'undeclared_var'

FINISHED WITH 1 ERROR(S)
```

Result: PASS — E2 correctly detected on line 9.

2.5 Test Case 5 — Invalid: Both Errors Present

File: test/test5_multi_errors.pas

Description:

Variable `a` is declared twice (E1), and identifier `ghost` is used without being declared (E2).

Expected behaviour:

- First `a` inserted successfully; second `a` triggers E1.
- Lookup of `ghost` fails, triggering E2.
- Both errors reported with correct line numbers.

Actual output:

```
[Scope 1] insert a : variable, integer, line 5
[Scope 1] insert b : variable, real, line 6
ERROR line 7: duplicate declaration 'a' in this scope
[Scope 1] lookup ghost -> NOT FOUND
ERROR line 9: undeclared identifier 'ghost'

FINISHED WITH 2 ERROR(S)
```

Result: PASS — Both E1 and E2 correctly detected.

3 Summary Table

#	File	Description	Errors Expected	Errors Found	Pass?
1	test1_valid_arith.pas	Valid arithmetic	0	0	
2	test2_valid_control.pas	Valid control flow	0	0	
3	test3_dup_decl.pas	Duplicate <code>count</code>	E1 (line 7)	E1 (line 7)	
4	test4_undecl_id.pas	Undeclared <code>undeclared_var</code>	E2 (line 9)	E2 (line 9)	
5	test5_multi_errors.pas	E1 + E2 together	E1+E2	E1+E2	

4 Limitations and Known Issues

1. **Grammar subset only.** The parser handles Pascal variable and constant declarations but does not yet support procedure/function bodies with separate scopes inside the body (i.e., no `procedure foo; var ...`). A function body would need an explicit `beginScope / endScope` pair; that hook exists in the code but is not wired to a grammar rule.
2. **Single comparison type.** The grammar allows `=`, `<`, `>`, `<=`, `>=`, `<>` in expressions for completeness, but type checking (integer vs. real comparison) is not performed — that belongs to a later semantic-analysis phase.
3. **Panic-mode recovery.** On a syntax error the parser skips one token. For deeply broken programs, this can produce spurious secondary errors (e.g., an undeclared error caused by a skipped declaration token).
4. **No `const` declaration support.** Pascal `const` sections are not part of the Lab-9 grammar and are therefore not parsed. The symbol table itself supports `KIND_CONST` entries; the parser just never generates them.
5. **Hash collisions are handled** via separate chaining but are not unit-tested explicitly (the 211-slot prime table keeps the load factor very low for typical small programs).